Learning Objectives

- Understand how function arguments and return values can be passed using the stack
- Distinguish the responsibilities of caller and callee
- Understand the stack layout during function calls
- Extend the code generator to generate function definitions and function calls
- Generate code for passing arguments and returning values
- Integrate the code generator from Session 10 into our compiler project

Preparation

Before we start working on calling conventions, we need to make sure that our compiler project contains the code generator developed in Session 10 and that the tools for our `hpc0` architecture are available.

1. **Add the code generator to the compiler project.**

   In Session 10, we started developing our code generator in `session10/codegen`. Copy all files from that directory into your `not-abc` directory, including `gen.abc`, `gen.hdr`, and the `xtest_gen*.abc` test programs.

2. **Make sure that the tools for `hpc0.isa` are installed.**

   In Session 10, we built and installed the virtual machine, assembler, and debugger from the `hpc0.isa`. If you have not installed the tools for a different ISA since then, there is nothing to do.

   Otherwise, repeat the procedure from Session 10:

   ```
   ulm-generator --fetch hpc0.isa
   sudo ulm-generator --install hpc0.isa
   ulm-generator --refman hpc0.isa
   ```

   The last command generates the instruction-set reference manual as `./hpc0/refman.pdf`.

Step 1 — From Instructions to Functions

Consider the following ABC program:

```
@ <stdio.hdr>

fn main()
{
    putchar('!');
    putchar('\n');
}
```

Using the function-call pattern developed in Session 10, we could write the following assembly program `ex1.s`:

```
    .text
    loadz   0,      %1

    call    main,   %3
    halt    0

main:
    subq    8,      %1,     %1
    movq    %3,     (%1)

    putc    '!'
    putc    '\n'

    movq    (%1),   %3
    addq    8,      %1,     %1
    ret     %3
```

This program produces the desired output, but it does not quite correspond to the ABC program above.

Instead of calling a function `putchar`, we directly use the `putc` instruction provided by our `hpc0` architecture. On the FPGA implementation, this instruction sends a byte to the serial interface.

However, input and output are inherently platform-dependent. On another system, output might require interacting with an operating system rather than directly accessing hardware.

This is exactly the kind of detail we want to hide behind a function interface. A program should be able to call `putchar` without knowing how characters are actually written on a particular platform. And there is another benefit for us: `putchar` takes an argument. Implementing it will give us an opportunity to extend our function-call pattern from Session 10 to support passing arguments.

Tasks

1. Save the assembly program above as `ex1.s`. Assemble and run it:

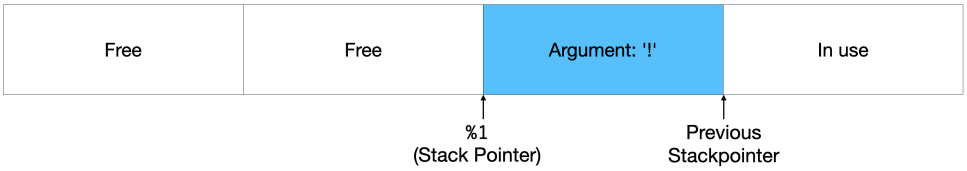
   ```
   ulmas -o ex1 ex1.s
   ulm ex1
   ```

2. **Passing an Argument to** `putchar`: We now want to replace the two `putc` instructions in `main` with calls to a function `putchar`.

   Let's start with the first character. Replace `putc '!'` with:

   ```
           subq    8,      %1,     %1
           loadz   '!',    %4
           movq    %4,     (%1)
           call    putchar,%3
           addq    8,      %1,     %1
   ```

   The **caller** reserves 8 bytes on the stack, stores the argument `'!'` there, calls `putchar`, and releases the stack space afterward. The stack looks as follows immediately before the call instruction is executed:

   Stack layout immediately before calling putchar. Each box represents 8 bytes of memory.

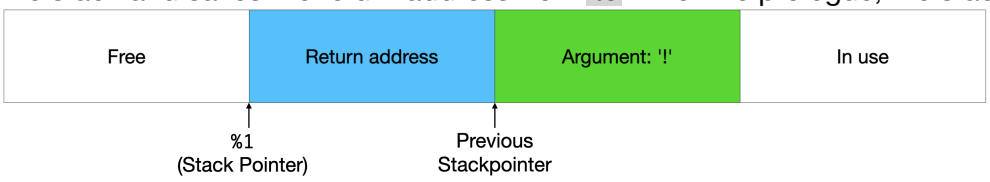
   Now consider the implementation of `putchar`:

   ```
   putchar:
           subq    8,      %1,     %1
           movq    %3,     (%1)

           movq    8(%1),  %4
           putc    %4

           movq    (%1),   %3
           addq    8,      %1,     %1
           ret     %3
   ```

   The function begins with the familiar prologue from Session 10: it reserves another 8 bytes on the stack and saves the return address from `%3`. After the prologue, the stack looks like this:

   Stack layout after executing the prologue of putchar. Each box represents 8 bytes of memory.

   Notice that the argument is now located at `8(%1)`, while the saved return address is at `(%1)`. The instruction `movq 8(%1), %4` retrieves the argument, and `putc %4` outputs the character.

   Finally, the epilogue restores the return address and releases the stack space reserved by the prologue. The `ret` instruction then returns control to the caller.

   Your task: Add the function `putchar` to `ex1.s` and replace the second `putc` instruction with another call to `putchar`, passing `'\n'` as its argument. Assemble and run the modified program.

Step 2 — Returning Values from Functions

In the previous step, we extended our function-call pattern to pass arguments on the stack. Now we want to extend it once more — this time to support return values. Consider the ABC program:

```
@ <stdio.hdr>

fn main()
{
    local ch: int = getchar();
    putchar(ch);
}
```

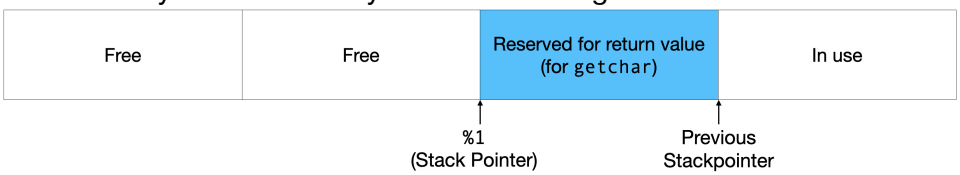
The function `getchar` takes no arguments but returns a character. As with arguments, we use the stack: the caller reserves 8 bytes, and the callee stores the return value there. In our assembly implementation, we will use a register to hold the local variable `ch`.

Tasks

1. Copy `ex1.s` to `ex2.s`. We will modify this copy in the following tasks.
2. Replace the first call to `putchar` in `main` with the following instructions:

```
        subq    8,      %1,     %1
        call    getchar,%3
        movq    (%1),   %4
        addq    8,      %1,     %1
```

   Notice that the caller reserves 8 bytes on the stack for the return value before calling `getchar`. The stack layout immediately before executing the call instruction is shown below.



Stack layout immediately before calling `getchar`. Each box represents 8 bytes of memory.

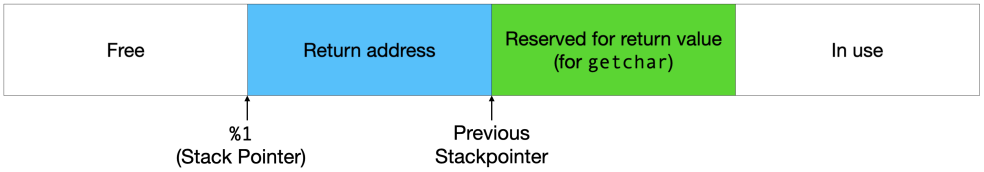
   After the call, the caller retrieves the return value from the stack into register `%4` and releases the reserved space.
3. Add the following implementation of `getchar` to `ex2.s`:

```
getchar:
        subq    8,      %1,     %1
        movq    %3,     (%1)

        getc    %4
        movq    %4,     8(%1)

        movq    (%1),   %3
        addq    8,      %1,     %1
        ret     %3
```

   The function begins with the familiar prologue, which saves the return address on the stack. The instruction `getc %4` then reads a character into register `%4`, and the following instruction stores it in the memory reserved by the caller.
   After the prologue, the stack looks as follows:



Stack layout after executing the prologue of `getchar`. Each box represents 8 bytes of memory.

   Notice that the return value is stored at `8(%1)`, while the saved return address is at `(%1)`. Unlike argument passing, the callee writes the value and the caller reads it.
4. Now replace the remaining call to `putchar` so that it prints the character returned by `getchar`, rather than a fixed character. Assemble and run `ex2.s` to verify that it reads a character and prints it back.

A Note on Calling Conventions

Passing arguments and returning values through the stack is not particularly difficult. The real challenge is bookkeeping: caller and callee must agree on where everything is stored.
In particular, we need to keep track of two perspectives:

- **Caller:** At which offsets do we store the arguments before a call, and where do we retrieve the return value afterward?
- **Callee:** At which offsets do we find the arguments and store the return value after entering the function?

To keep things manageable, we will adopt two simplifying conventions:
- Every value occupies 64 bits (8 bytes).
- Every function has a return-value slot, even if it does not return a value.

This eliminates some special cases. We can now use the same stack layout for every function call.

Step 3 — Putting It All Together

Now consider a function that takes two arguments and returns their sum:

```
fn sum(a: u64, b: u64): u64
{
    return a + b;
}

fn main(): u64
{
    return sum(12, 30);
}
```

Our goal is to implement this program in assembly using the calling convention developed in the previous steps. The program should return `42` as its exit status.

Tasks

1. **Calling `main` and terminating the program.** Create a new assembly file `ex4.s`. We begin with the startup code that initializes the stack pointer and calls `main`. Unlike in our previous examples, we now assume that `main` returns a value. Using the pattern from Step 2, we obtain:

```
        .text
        loadz   0,      %1

        subq    8,      %1,     %1
        call    main,   %3
        movq    (%1),   %4
        addq    8,      %1,     %1
        halt    %4
```

   The startup code reserves space for the return value of `main`, retrieves it after the call, and uses it as the program's exit code. **Looking ahead:** Our code generator will eventually produce this entire sequence in `genInit()`. Since it already includes the instructions for terminating the program, we will no longer need `genExit()`.

2. **Defining `main`.** Next, add the following stub for `main` to `ex4.s`:

```
main:
        subq    8,      %1,     %1
        movq    %3,     (%1)

        /* function body here */

        movq    %4,     8(%1)

        movq    (%1),   %3
        addq    8,      %1,     %1
        ret     %3
```

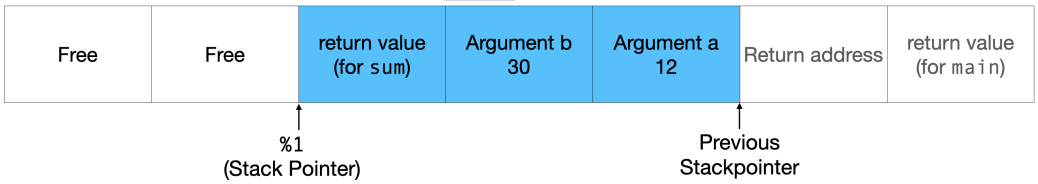
   The prologue and epilogue follow the familiar pattern from Session 10. As in Step 2, we use the stack to return a value to the caller. For now, we assume that the function body leaves its result in register `%4`. The instruction `movq %4, 8(%1)` then stores this value in the return-value slot reserved by the caller, just before the epilogue restores the return address and releases the stack space.

3. **Calling sum with two arguments.** Now replace `/* function body here */` in `main` with the following instructions:

```
subq    24,     %1,     %1
loadz   12,     %4
movq    %4,     16(%1)
loadz   30,     %4
movq    %4,     8(%1)
call    sum,    %3
movq    (%1),   %4
addq    24,     %1,     %1
```

The caller reserves 24 bytes on the stack: 8 bytes for the return value and 8 bytes for each argument. It then stores the arguments `12` and `30` at offsets 16 and 8, respectively.

Immediately before executing the `call` instruction, the stack looks as follows:



After the call, the caller retrieves the return value from `(%1)` into register `%4` and releases the 24 bytes reserved for the function call. **Notice:** The order and offsets of the arguments are part of our calling convention. The implementation of sum must follow exactly the same convention.

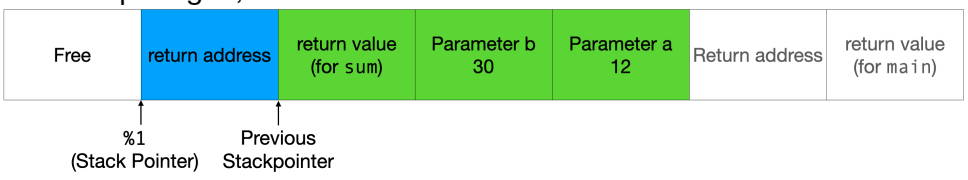
4. **Defining sum.** Finally, add the following implementation of sum to `ex4.s`:

```
sum:
        subq    8,      %1,     %1
        movq    %3,     (%1)

        movq    24(%1), %4
        movq    16(%1), %5
        addq    %4,     %5,     %4
        movq    %4,     8(%1)

        movq    (%1),   %3
        addq    8,      %1,     %1
        ret     %3
```

The function begins with the familiar prologue, which saves the return address on the stack. After the prologue, the stack looks as follows:



Notice how the prologue shifts the offsets by 8 bytes. The return address is now at `(%1)`, the return-value slot at `8(%1)`, and the two arguments at `16(%1)` and `24(%1)`.

The function loads the arguments into registers `%4` and `%5`, adds them, and stores the result in the return-value slot reserved by the caller. Finally, the epilogue restores the return address and releases the stack space before `ret %3` returns control to the caller.

Assemble and run `ex4.s`. The program should terminate with exit code `42`.

Looking Ahead — From Assembly Patterns to Code Generation

We now have a calling convention that supports function arguments and return values. The assembly patterns are somewhat tedious to write by hand, but we only need to implement them once in our code generator.

There is, however, one detail worth noticing in the stack diagrams above: the caller and callee use different offsets to access the same arguments and return-value slot. We can hide this inconvenience behind functions in our code generator, but we will eventually revisit our calling convention and improve the underlying pattern itself.

Step 4 — From Calling Conventions to Code Generation (Preparation)

We have developed a calling convention for passing arguments and returning values. Now it is time to put these assembly patterns into our code generator.

From now on, we will work in the `not-abc` directory of our compiler project.

Tasks

1. **Extending `genInit()`.** In Session 10, `genInit()` only initialized the stack pointer. We now want it to generate the startup code developed in Step 3, including the call to `main` and the use of its return value as the program's exit code.
   In addition, `genInit()` should generate implementations of `putchar` and `getchar`, so that programs generated by our compiler can use these functions.
   Modify `genInit()` in `gen.abc` so that it produces the following assembly code:

```
        .text
        loadz   0,      %1

        subq    8,      %1,     %1
        call    main,   %3
        movq    (%1),   %4
        addq    8,      %1,     %1
        halt    %4

putchar:
        subq    8,      %1,     %1
        movq    %3,     (%1)
        movq    16(%1),         %4
        putc    %4
        movq    (%1),   %3
        addq    8,      %1,     %1
        ret     %3

getchar:
        subq    8,      %1,     %1
        movq    %3,     (%1)
        getc    %4
        movq    %4,     8(%1)
        movq    (%1),   %3
        addq    8,      %1,     %1
        ret     %3
```

   Notice that `putchar` now follows the calling convention established in Step 3. Since every function call reserves a return-value slot, its argument is located at `16(%1)` after the prologue, rather than at `8(%1)` as in Step 1.
   This is quite a bit of typing — and another opportunity to practice escaping `%` and other special characters in `printf` format strings. But if you know how to use your editor efficiently, that should not stop you. ;-)
   You can test your implementation using the test programs from Session 10, for example:

```
abc -o xtest_gen_01 xtest_gen_01.abc gen.abc
./xtest_gen_01
```

2. **Breaking the Interface: Removing `genExit()`.**

   Our new implementation of `genInit()` already generates the instructions for terminating the program. Therefore, `genExit()` is no longer needed.

   However, changing an interface usually requires more than modifying its implementation. We also need to update all code that depends on it. In our case, both `xtest_gen_01.abc` and `xtest_gen_02.abc` still call `genExit()`.

   Fortunately, our compiler project is still at an early stage, so we can keep things simple:
   - Remove the declaration of `genExit()` from `gen.hdr` and its definition from `gen.abc`.
   - Delete `xtest_gen_02.abc`.
   - Remove the call to `genExit()` from `xtest_gen_01.abc`.

   We will continue adapting `xtest_gen_01.abc` in the following tasks.

   The lesson: Even a small change to an interface can require changes in several other places. The larger a software project becomes, the more important it is to keep track of such dependencies.

3. **Preparing the Code Generator for Function Calls.**
We now extend our code generator so that it can generate assembly code for the program from Step 2, which reads a character and prints it back. Modify `xtest_gen_01.abc` as follows:

```
@ "gen.hdr"

fn main()
{
    genInit();

    genFuncDefBegin("main");
    genCallBegin("getchar", 0);
    local ch: GenReg = genCallEnd();

    genCallBegin("putchar", 1);
    genCallAddArg(ch, 0);
    genReleaseReg(genCallEnd());
    genReleaseReg(ch);

    genFuncDefEnd();
}
```

Next, extend the interface in `gen.hdr` with the following declarations:

```
extern fn genFuncDefBegin(name: ->readonly char);
extern fn genFuncDefEnd();

extern fn genCallBegin(fnName: ->readonly char, numArgs: unsigned);
extern fn genCallAddArg(argReg: GenReg, argPos: unsigned);
extern fn genCallEnd(): GenReg;
```

Finally, add the corresponding stub definitions to `gen.abc`.

```
fn genFuncDefBegin(name: ->readonly char)
{
   assert(0 && "TODO");
}

fn genFuncDefEnd()
{
   assert(0 && "TODO");
}

struct CallInfo
{
    fnName:  ->readonly char;
    numArgs: unsigned;
};

global callInfo: CallInfo;

fn genCallBegin(fnName: ->readonly char, numArgs: unsigned)
{
    assert(0 && "TODO");
}

fn genCallAddArg(argReg: GenReg, argPos: unsigned)
{
    assert(0 && "TODO");
}

fn genCallEnd(): GenReg
{
    assert(0 && "TODO");
    local retVal: GenReg = getReg();
    return retVal;
}
```

Compile and run the modified test program:

```
abc -o xtest_gen01 xtest_gen_01.abc gen.abc
./xtest_gen01
```

In the following steps, we will implement these functions one by one. Time for another round of assertion chasing! ;-)

Step 5 — From Calling Conventions to Code Generation (Implementation)

We are now ready to replace the stubs from Step 4, one by one, using the assembly patterns developed in Step 3.

Tasks

1. **Implementing `genFuncDefBegin()`.** The first assertion comes from `genFuncDefBegin()`. Recall from Session 10 that whenever a code generator function emits instructions for the text segment, it should first call `setSegment(TEXT)`. This ensures that the generated instructions are placed in the correct segment, without unnecessarily repeating the `.text` directive.

   Now replace the stub for `genFuncDefBegin()` with:

   ```
   fn genFuncDefBegin(name: ->readonly char)
   {
       setSegment(TEXT);
       printf("%s:\n", name);
       printf("\tsubq \t8, \t%%1, \t%%1\n");
       printf("\tmovq \t%%3, \t(%%1)\n");
   }
   ```

   The function generates the label and the familiar function prologue: it reserves 8 bytes on the stack and saves the return address from `%3`.

   From now on, after each task, recompile and run `xtest_gen01` again. The next assertion will tell us which function to implement next.

2. **Implementing `genCallBegin()`.** The next assertion comes from `genCallBegin()`. Before implementing it, let's take a closer look at how our interface represents the assembly pattern for function calls developed in Step 3.

   We split the generation of a function call into three stages:

   1. `genCallBegin(fnName, numArgs)` reserves space on the stack for the arguments and the return value. It also receives the name of the function we want to call.
   2. `genCallAddArg(argReg, argPos)` stores an argument on the stack. It needs to know which register contains the argument and at which argument position it belongs.
   3. `genCallEnd()` generates the actual function call, retrieves the return value from the stack into a free register, releases the stack space, and returns the register holding the result.

   Notice that the actual call instruction is generated only by `genCallEnd()`. Until then, we need to remember which function we are going to call and how many arguments it expects.

   For this purpose, we introduced the following structure and global variable in Step 4:

   ```
   struct CallInfo
   {
       fnName:  ->readonly char;
       numArgs: unsigned;
   };

   global callInfo: CallInfo;
   ```

   With this information, implementing `genCallBegin()` is straightforward:

   ```
   fn genCallBegin(fnName: ->readonly char, numArgs: unsigned)
   {
       setSegment(TEXT);
       callInfo.fnName = fnName;
       callInfo.numArgs = numArgs;

       printf("\tsubq \t%u, \t%%1, \t%%1\n", 8*(callInfo.numArgs + 1));
   }
   ```

   The expression `8*(callInfo.numArgs + 1)` calculates the required stack space: 8 bytes for each argument and another 8 bytes for the return value.

3. **Implementing `genCallEnd()`.** The next assertion comes from `genCallEnd()`. Since `getchar` takes no arguments, `genCallAddArg()` was not called. Using the information stored in `callInfo` by `genCallBegin()`, `genCallEnd()` should now generate:

   ```
   call    getchar, %3
   movq    (%1),    %4
   addq    8,       %1,     %1
   ```

4. **Implementing `genCallAddArg()`.** The next assertion comes from `genCallAddArg()`. We have now reached the call to `putchar`, which takes one argument.

   Recall that `genCallBegin()` has already reserved space on the stack for the return value and all arguments. The task of `genCallAddArg()` is to store an argument in its designated stack location.

   The parameter `argReg` identifies the register containing the argument, while `argPos` specifies its position in the argument list. Since the return-value slot occupies the first 8 bytes, the stack offset for an argument is `8 * (callInfo.numArgs - argPos)`.

   This gives us:

   ```
   fn genCallAddArg(argReg: GenReg, argPos: unsigned)
   {
       setSegment(TEXT);
       printf("\tmovq \t%%%d, \t%u(%%1)\n", argReg, 8 * (callInfo.numArgs - argPos));
   }
   ```

   The parameter `argReg` identifies the register containing the argument, while `argPos` specifies its position in the argument list. Since the return-value slot occupies the first 8 bytes, the stack offset is `8 * (callInfo.numArgs - argPos)`.
   For example, when calling `sum(12, 30)` from Step 3, argument 0 is stored at `16(%1)` and argument 1 at `8(%1)`. Recall that `argPos` starts at zero.

5. **Implementing `genFuncDefEnd()`.** The last assertion comes from `genFuncDefEnd()`. We are almost done!

   This function generates the familiar epilogue and return instruction. Regardless of which function we are defining, it should always produce:

   ```
   movq    (%1),   %3
   addq    8,      %1,     %1
   ret     %3
   ```

   Implement `genFuncDefEnd()` accordingly.

   With this final piece in place, our test program should compile and run without triggering any assertions.

   More importantly, **we can now generate an assembly program that actually runs!**

   Try the complete workflow:

   ```
   abc -o xtest_gen01 xtest_gen_01.abc gen.abc
   ./xtest_gen01 > foo.s
   ulmas -o foo foo.s
   ulm foo
   ```

   Enter a character, for example `A`. The program should read it and print it back.

Looking Ahead — Nested Function Calls

The assembly program generated by our code generator corresponds to the ABC program from Step 2. However, when writing the program by hand, we would probably prefer the more compact version:

```
@ <stdio.hdr>

fn main()
{
    putchar(getchar());
}
```

Unfortunately, our code generator does not yet support nested function calls.

We will investigate this problem in more detail in a later session. For now, let's write a test program that attempts to generate code for nested function calls.

To our horror, we will discover that our code generator produces incorrect assembly without reporting any error!

The truly shocking part is not the incorrect code — incomplete features are perfectly normal during development. The real problem is that nothing warns us that an unsupported feature may silently produce incorrect results.

Step 6 — Detecting Unsupported Nested Function Calls

1. Copy `xtest_gen_01.abc` to `xtest_gen_02.abc` and modify it as follows:

```
@ "gen.hdr"

fn main()
{
    genInit();

    genFuncDefBegin("main");

    genCallBegin("putchar", 1);

    genCallBegin("getchar", 0);
    local ch: GenReg = genCallEnd();
    genCallAddArg(ch, 0);
    genReleaseReg(ch);

    genReleaseReg(genCallEnd());

    genFuncDefEnd();
}
```

   This should generate code corresponding to `putchar(getchar())`.

   Compile and run the test:

```
abc -o xtest_gen02 xtest_gen_02.abc gen.abc
./xtest_gen02
```

   Inspect the generated assembly. Something is clearly wrong: `getchar` is called twice, while `putchar` is never called! Yet our code generator reports no error.

2. **Detecting unsupported nested calls.**

   We now want to ensure that `genCallBegin()` can only be called when code generation for the previous function call has been completed. Fortunately, we can use `callInfo.fnName` to keep track of whether a function call is currently being generated.

   Modify `genCallBegin()`:

```
fn genCallBegin(fnName: ->readonly char, numArgs: unsigned)
{
    setSegment(TEXT);
    assert(!callInfo.fnName);
    callInfo.fnName = fnName;
    callInfo.numArgs = numArgs;

    printf("\tsubq \t%u, \t%%1, \t%%1\n", 8*(callInfo.numArgs + 1));
}
```

   The assertion ensures that no other function call is currently being generated. Of course, we must also mark a function call as completed. Modify `genCallEnd()` accordingly:

```
fn genCallEnd(): GenReg
{
    setSegment(TEXT);
    local retVal: GenReg = getReg();
    printf("\tcall \t%s, %%3\n", callInfo.fnName);
    printf("\tmovq \t(%%1), \t%%%u\n", retVal);
    printf("\taddq \t%u, \t%%1, \t%%1\n", 8 * (1 + callInfo.numArgs));
    callInfo.fnName = nullptr;
    return retVal;
}
```

3. **Testing our assertion.** Recompile and run both test programs. The original `xtest_gen01` should still work, while `xtest_gen02` should now terminate with an assertion failure.

   We have not implemented nested function calls, but we have made an important improvement: our code generator now detects an unsupported use instead of silently producing incorrect code.

   That is progress, too. ;-)